

GUÍA RESUMEN — Widgets de Flutter

Resumen integral de widgets de Flutter (Material 3) sin volver a leer el módulo completo. Hilo conductor: Fundamentos → Básicos y atomización → Layout y navegación → Formularios → Listas → Estado y ciclo de vida → Animaciones → Composición → Renderización → Performance → Migración de APIs deprecadas.

0 Fundamentos: los 3 árboles y la regla de oro

WIDGET • ELEMENT • RENDER

En Flutter **todo es un widget**: una descripción inmutable de la UI. El modelo es **declarativo** — describes cómo debe verse la UI para un estado y Flutter la lleva ahí.

Widget configuración, barato de crear/descartar	Element instancia mutable que Flutter reutiliza	RenderObject layout + paint en pantalla
---	---	---

Stateless vs Stateful

	StatelessWidget	StatefulWidget
Estado mutable	No	Sí (vive en la clase <code>State</code>)
Reconstruye por	Nuevas propiedades / contexto	Propiedades + <code>setState()</code>
Usa cuando	Solo depende de props y contexto	Necesita estado efímero interno

Regla de oro: ¿el widget cambia con el tiempo? No → `StatelessWidget` · Solo por datos externos → `StatelessWidget` + `Stream` / `Future` · Estado efímero interno → `StatefulWidget`.

Claves rápidas

- **BuildContext** = el "lugar" del widget en el árbol (acceso a `Theme.of`, `Navigator`, `MediaQuery.sizeOf`).
- **const** en todo widget que no cambie → Flutter se salta su reconstrucción.
- **Keys:** `ValueKey` (identificador), `ObjectKey` (objeto), `UniqueKey` (forzar re-creación), `GlobalKey` (acceder desde fuera).

1 Widgets básicos y atomización

LOS BLOQUES DE CONSTRUCCIÓN

Categoría	Widgets
Texto	<code>Text</code> , <code>RichText</code> , <code>SelectableText</code> , <code>DefaultTextStyle</code>
Imagen	<code>Image.asset/.network</code> , <code>CircleAvatar</code> , <code>CachedNetworkImage</code>
Espaciado	<code>SizeBox</code> > <code>Container</code> ; <code>ColoredBox</code> / <code>DecoratedBox</code> (ligeros)
Botones M3	<code>FilledButton</code> , <code>FilledButton.tonal</code> , <code>OutlinedButton</code> , <code>TextButton</code> (<code>ElevatedButton</code> = legacy M2)
Chips	<code>Chip</code> , <code>InputChip</code> , <code>FilterChip</code> , <code>ChoiceChip</code> , <code>ActionChip</code>
Contenedores	<code>Card</code> , <code>Card.filled</code> , <code>Card.outlined</code> , <code>Badge</code>

Atomización (atomic design)

Átomos (Text, Icon, Chip, Botones) → Moléculas (Card, ListTile, AppBar, NavigationBar) → Organismos (Formulario, Lista, Scaffold) → Plantillas → Páginas.

Crea widgets atómicos propios (`AppText`, `UserAvatar`) para consistencia; hazlos `const` y sin estado cuando sea posible.

Row / Column / Stack

`Row` = eje horizontal · `Column` = eje vertical. `mainAxisAlignment` sobre el eje principal, `crossAxisAlignment` sobre el secundario. `Expanded` reparte espacio proporcional (flex), `Flexible` permite encoger. `Stack` + `Positioned` superponen capas.

Scaffold (M3)

Barra inferior: `NavigationBar` + `NavigationDestination` (M3). Drawer: `NavigationDrawer`. `BottomNavigationBar` y `Drawer` son legacy M2.

Navegación

API	Uso
Navigator 1.0	<code>Navigator.push/pop</code> imperativo, rutas por clase
GoRouter (recomendada)	Rutas declarativas con <code>GoRoute</code> , params, redirects; <code>context.go / push / pop</code> ; pubspec: <code>go_router ^17.5.0</code>
PopScope	Intercepta el botón atrás (reemplaza <code>WillPopScope</code>)

Responsive: usa `MediaQuery.sizeOf/widthOf/heightOf` (no `MediaQuery.of().size`) para decidir por pantalla, y `LayoutBuilder` para decidir por el espacio del padre.

3 Interacción y Formularios

Formularios

`Form(key: GlobalKey<FormState>)` + `TextFormField validator:` · `validate()` corre todos los validators; `save()` dispara `onSaved`. Controladores (`TextEditingController`) siempre en `dispose()`.

Selección y controles M3

Control	Nota clave
<code>DropdownButtonFormField</code>	Usa <code>initialValue</code> (no <code>value</code> , deprecado 3.35)
<code>DropdownMenu</code>	Alternativa M3 con búsqueda
<code>Radio</code>	Dentro de <code>RadioGroup</code> ; <code>groupValue</code> / <code>onChanged</code> deprecados en Radio
<code>Switch</code>	Usa <code>activeThumbColor</code> (no <code>activeColor</code>)
<code>SegmentedButton</code>	Selección exclusiva estilizada
<code>Slider</code> / <code>RangeSlider</code>	Rangos con <code>divisions</code> y <code>labels</code>

Patrón de formulario sin librerías

`Form(key: _formKey)` → campos `TextFormField` con `validator`
Botón: `FilledButton` habilitado según estado
Estado local: `ValueNotifier<bool>(_enviando)` + `ValueListenableBuilder`

Adelanto módulo 16: el mismo árbol de widgets se mantiene con `BlocProvider` + `BlocListener` + `BlocBuilder`. La estructura del formulario no cambia, solo quién gestiona el estado.

Widget	Cuándo
<code>ListView</code>	Pocos hijos conocidos
<code>ListView.builder</code>	Listas grandes: construye solo lo visible (preferido)
<code>ListView.separated</code>	Con separadores entre ítems
<code>GridView</code>	Cuadrículas (count / builder)
<code>CustomScrollView</code> + Slivers	Combinar AppBar colapsable + lista + grid en un scroll
<code>CarouselView</code> (3.35+)	Carrusel de tarjetas con snapping
<code>AnimatedList</code> / <code>SliverAnimatedList</code>	Inserciones/eliminaciones animadas
<code>ReorderableListView</code>	Orden por drag & drop (keys obligatorias)

Pull-to-refresh: `RefreshIndicator(onRefresh: )` con una función `Future`. **Infinite scroll:** `ScrollController` + listener que detecta el final (`pixels ≥ maxScrollExtent - 200`). **Nota:** `cacheExtent` está deprecado desde 3.44.

5 Datos, Estado y Ciclo de Vida

FLUJO UNIDIRECCIONAL

Los datos viajan **hacia abajo**; los callbacks **hacia arriba** (lifting state up).

Ciclo de vida StatefulWidget

```
constructor → initState → didChangeDependencies → build
(padre cambia) didUpdateWidget → build
(desmonta) dispose
// setState() programa una reconstrucción del subárbol
```

Widgets reactivos

Widget	Escucha	Uso
<code>FutureBuilder</code>	Future	Carga única (guardar el future en variable para reintentar)
<code>StreamBuilder</code>	Stream	Emisiones continuas (opcional <code>initialData</code>)
<code>ValueListenableBuilder</code>	ValueNotifier	Estado local simple
<code>ListenableBuilder</code>	cualquier Listenable	Objetos ChangeNotifier, Animations

mounted vs context.mounted: `mounted` (en el `State`) protege `setState` tras un `await`; `context.mounted` (en el `BuildContext`) protege operaciones con context (navegación, `SnackBar`, `context.read`).

6 Animaciones

IMPLÍCITAS VS EXPLÍCITAS

Implícitas (cambia el target, Flutter interpola)

`AnimatedContainer`, `AnimatedOpacity`, `AnimatedPadding`, `AnimatedSwitcher`, `AnimatedPositioned`, `AnimatedAlign`, `AnimatedSlide/Scale/Rotation`, `TweenAnimationBuilder`. Personalizables con `AnimationStyle`.

Explícitas (control fino)

`AnimationController` (+ `SingleTickerProviderStateMixin`) → `Tween` → `CurvedAnimation` → `AnimatedBuilder`. Con `AnimationStatusListener` para repetir/invertir y `Interval` para animaciones escalonadas.

Extras

- **Hero:** transición compartida entre pantallas con el mismo `tag`.
- **CustomPainter:** dibujo vectorial (`paint` + `shouldRepaint`).
- **Lottie/Rive:** animaciones JSON de After Effects.

Reglas: implícitas primero · ~300ms en interacciones táctiles · no animes opacity+size juntos · usa `FadeTransition` en loops (no `Opacity`).

- **No heredes widgets:** compón con parámetros (`AccionBoton` envuelve un `FilledButton`).
- **Slots:** expón `child` (contenido fijo) o `builder` (contenido que depende del contexto).
- **Widgets privados anidados** (`_Body` , `_InfoSection`) → sub-árboles `const` que se reconstruyen independientemente.
- **Una responsabilidad por widget** (Page / View / Card / Tile / Form / Dialog...).
- **Builder pattern:** crear un contexto hijo para SnackBars y similares.

InheritedWidget: mecanismo interno de `Theme` , `MediaQuery` y `BlocProvider` . Entiende cómo funciona, no lo reimplementes.

8 Patrones de Renderización

LOADING · ERROR · EMPTY · DATA

Renderizado condicional

Ternario → `collection-if` / `collection-for` → `Visibility` (con `maintainSize` / `maintainState`).

Estados con clases selladas (Dart 3)

```
sealed class AsyncState<T>
  | AsyncLoading<T>
  | AsyncError<T> (message)
  | AsyncData<T> (data)

UI: switch (estado) → loading spinner / error + reintentar / empty / builder(data)
```

Scroll y vistas

- **Slivers:** AppBar colapsable + listas + grids en `CustomScrollView` .
- **NestedScrollView:** header que colapsa con Tabs en el body.
- **IndexedStack:** mantiene vistas montadas (no pierde estado). **PageView:** swipe.
- **Shimmer/Skeleton:** placeholders animados mientras carga.
- **Overlay/Popup:** bottom sheets, tooltips, PopupMenuButton.
- **Callback pattern:** el hijo notifica al padre (inversión de control).

9 Rendimiento y Buenas Prácticas

REBUILDS MÍNIMOS

Práctica	Qué evita
<code>const</code> en widgets que no cambian	Reconstrucciones innecesarias
Extraer sub-widgets <code>const</code>	Rebuilds en cascada
<code>ListView.builder</code>	Construir items invisibles
<code>RepaintBoundary</code>	Repintar mapas/videos/canvas al hacer scroll
Callbacks como métodos (no closures)	Nuevas closures cada build
<code>AnimatedBuilder</code> en animaciones	setState por frame
<code>ValueKey</code> en listas dinámicas	Estado confundido al reordenar
Imágenes con <code>width/height</code> + caché	Decodificaciones gigantes
<code>dispose()</code> de controladores	Memory leaks

Herramientas: Flutter DevTools → Rebuild Counts, Track widget rebuilds, Performance overlay, Memory/CPU.

Deprecado	Reemplazo	Desde
<code>DropDownButtonFormField.value</code>	<code>initialValue</code>	3.35
<code>Radio.groupValue/onChanged</code>	Envolver en <code>RadioGroup</code>	3.32
<code>Switch.activeColor</code>	<code>activeThumbColor</code>	3.35
<code>MediaQuery.of(context).size</code>	<code>MediaQuery.sizeOf/widthOf/heightOf</code>	3.35
<code>MaterialState</code>	<code>WidgetState</code>	3.22
<code>WillPopScope</code>	<code>PopScope</code>	3.24
<code>Color.withOpacity</code>	<code>withValues(alpha:)</code>	3.27
<code>findChildIndexCallback</code>	<code>findItemIndexCallback</code>	3.41
<code>cacheExtent</code> en viewports	Automático (no declarar)	3.44
<code>ExpansionTileController</code>	<code>ExpansibleController</code>	3.32
<code>AppBar</code>	<code>OverflowBar</code>	3.24

Widgets nuevos a aprovechar: `SegmentedButton`, `DropDownMenu`, `SearchBar` / `SearchAnchor`, `MenuAnchor`, `CarouselView` (3.35+), `Card.filled/outlined`, `ListenableBuilder`, `Badge`, `NavigationBar`, `RadioGroup`.

A Anexo · Índice del módulo (para profundizar)

Necesitas	Archivo
Widget/Element/Render, BuildContext, keys, const	01-fundamentos-widgets.md
Text, Image, Icon, botones M3, chips, Card, Badge	02-widgets-basicos-y-atomizacion.md
Row/Column/Stack, Scaffold, Navigator, GoRouter, PopScope	03-layout-y-navegacion.md
TextField/Form, validación, RadioGroup, DropdownMenu, SegmentedButton	04-interaccion-y-formularios.md
ListView/GridView, slivers, CarouselView, pull-to-refresh, paginación	05-listas-y-scroll.md
Ciclo de vida, mounted, FutureBuilder/StreamBuilder/ListenableBuilder	06-datos-estados-y-ciclo-de-vida.md
Animaciones implícitas/explotas, Hero, CustomPainter, Lottie	07-animaciones.md
Composición vs herencia, slots, InheritedWidget, naming	08-estrategias-composicion.md
Estados sealed, loading/error/empty, slivers, overlays	09-patrones-renderizacion.md
const, RepaintBoundary, keys, DevTools, checklist	10-perf-y-buenas-practicas.md
+150 widgets por categoría, Material 3, Cupertino	11-arsenal-completo-widgets.md

Widgets de Flutter en una línea: Fundamentos (3 árboles + const + keys) → Básicos atómicos → Layout + GoRouter → Formularios M3 → Listas eficientes → Estado y ciclo de vida → Animaciones → Composición → Renderización → Performance. · Regla de oro: cada widget con una responsabilidad, y todo lo que no cambia debe ser `const`.